Introduction to Software Engineering

Project Proposal

Prepared by:

Nguyễn Thái Cường (24127336)

Nguyễn Thanh Tùng (24127583)

Đỗ Trương Khoa (24127423)

K'Vớn (24127593)

Đào Hoàng Phúc (24127496)

Instructor:

Dr. Trần Duy Hoàng

MSc. Trương Phước Lộc

MSc. Phạm Hoàng Hải

Software Engineering Department

Faculty of Information and Technology

University of Science

Hồ Chí Minh city, June 30, 2026

Table of Contents

Project Proposal

Objectives

This document focus on the following topics:

- Completing the Project Proposal document with the following sections:
  - Preliminary Problem Statement
  - Proposed Solution
  - Development Plan
  - Human Resources & Costing Plan
- Understanding the Project Proposal document.

1 Member Contribution Assessment

❖ **Group ID:** 09

ID	Name	Email	Contribution (%)
24127336	Nguyễn Thái Cường	ntcuong2434@clc.fitus.edu.vn	100 %
24127583	Nguyễn Thanh Tùng	nttung2410@clc.fitus.edu.vn	100 %
24127423	Đỗ Trương Khoa	dtkhoa2411@clc.fitus.edu.vn	100 %
24127593	K'Vớn	kvon2408@clc.fitus.edu.vn	100 %
24127496	Đào Hoàng Phúc	dhphuc2430@clc.fitus.edu.vn	100 %

2 Preliminary Problem Statement

The demand for sports activities is consistently growing, yet the traditional process of finding, booking, and managing sports venues remains manual and inefficient. Currently, users often have to rely on direct phone calls to confirm bookings, which is time-consuming and lacks convenience. Furthermore, players struggle to find high-quality venues that fit their specific needs regarding location, budget, and sport type. Another significant challenge is the lack of a platform for solo players or teams to connect and arrange friendly matches.

On the operational side, venue owners face difficulties in efficiently managing their schedules, verifying customer identities, and tracking business performance. The absence of automated notifications and detailed revenue analytics by sport type hinders their ability to optimize operations

To address these fundamental issues, we propose the development of **SPOT (Sport Pitch Online Ticketing)** - a comprehensive **Online Sports Venue Booking Management System**. The **SPOT** platform aims to digitize and streamline the entire booking ecosystem, allowing users to independently **book venues online**, view real-time **available time slots**, and utilize **advanced search filters**. To foster sportsmanship, **SPOT** will also introduce a dedicated **matchmaking feature** connecting different sports groups and solo players.

For administrators and venue owners, **SPOT** will provide robust management tools, including **automated booking notifications**, detailed **customer verification**, and **monthly revenue statistics**. To mitigate operational risks and revenue loss from unfulfilled appointments, the platform will implement a **Smart No-Show Prevention System**. To ensure a high-quality user experience and safety, the platform will enforce strict security protocols (such as **OTP** for sensitive actions and **brute-force prevention**), maintain a fast response time of under 3 seconds, and implement **role-based access control**. Finally, **SPOT** will differentiate itself by leveraging advanced AI capabilities, including a **personalized recommendation engine** based on user activity and an **NLP-powered virtual assistant** for a quick, hands-free booking experience

To successfully implement the proposed architecture and ensure system maintainability, SPOT will be built upon a modern and highly scalable technology stack. The client-side interfaces will be developed using **React and Next.js** for the Web application and Admin Console, alongside **React Native** to deliver a seamless, high-performance cross-platform Mobile App. On the server side, the API Gateway and Core Business Services will be powered by **Node.js and Express**, ensuring fast, non-blocking operations.

Data persistence will rely on **PostgreSQL** for robust transactional integrity, complemented by **Redis** for high-speed caching to guarantee the sub-3-second response time requirement. Crucially, the AI-driven components—specifically the Smart No-Show Prevention System and the Recommendation Engine—will be developed using **Python** (incorporating libraries like scikit-learn and XGBoost) to effectively handle complex data classification, decision-making logic, and predictive modeling. The NLP virtual assistant will integrate with state-of-the-art LLM APIs such as **Gemini / GPT**. Finally, the entire ecosystem will be containerized using **Docker** and hosted on **Vercel** to streamline continuous deployment and cloud operations.

3 Proposed Solution

3.1. Software

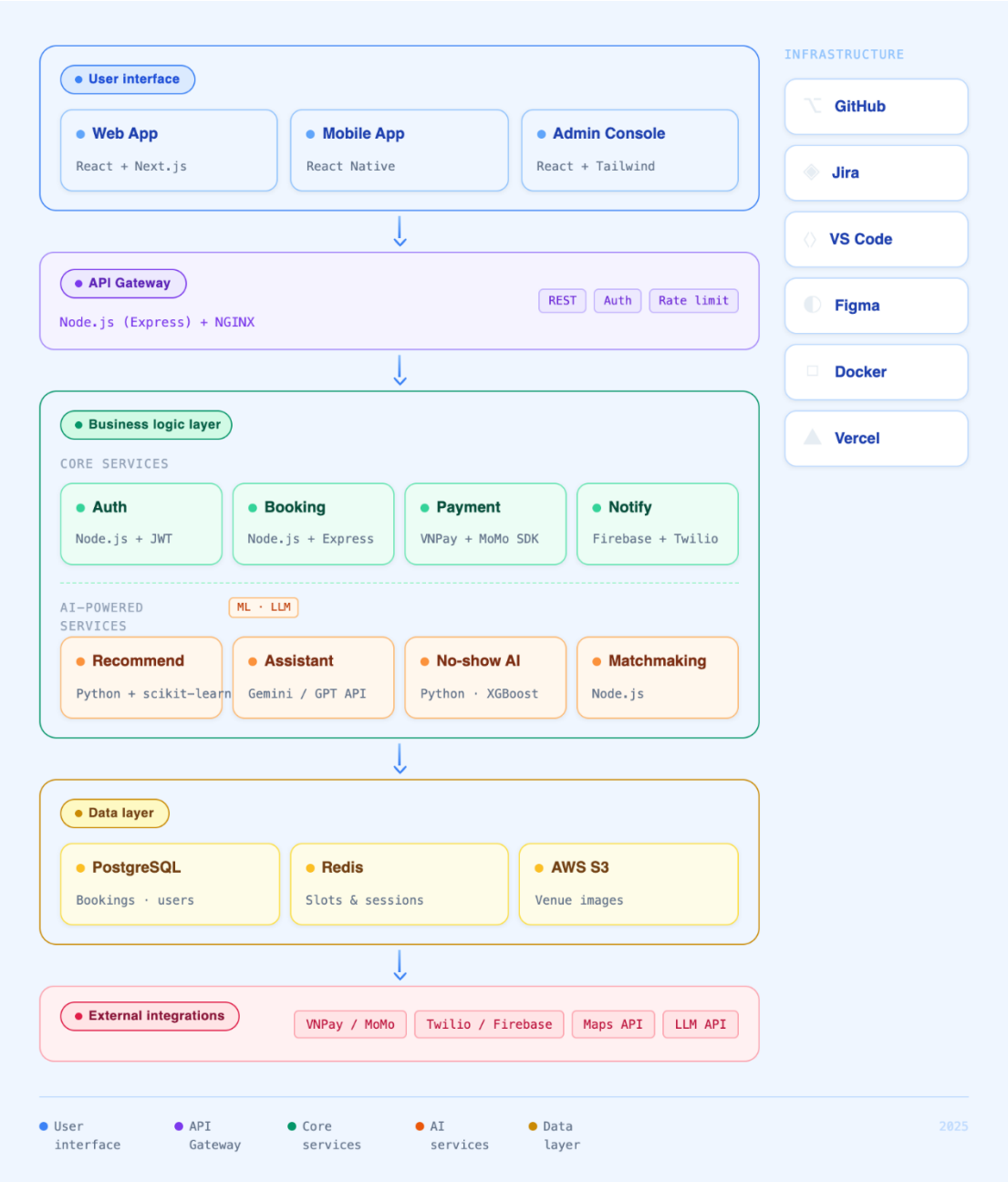
3.1.1. Features

Demand	*Request*
As an administrator, I want to **assign** specific login permissions to **different user types** so that they can perform designated functions	Login / Register
As a customer, I want to **re-confirm** my password or enter **an OTP** before performing **sensitive actions** such as changing my email, phone number, …	Security
As a venue owner/ an administrator, I want the ability to **view details**, **modify**, or **cancel any customer's booking** so that I can **effectively manage** the venue schedule and **assist** customers when needed	Security
As an administrator, I want to **limit** the number of consecutive **failed login attempts** to **prevent** brute-force attacks	Security
As a user, I want to **rate and review** the venue after using it so that I can share my experience with others	Reviews and Comments
As a venue owner, I want to **receive a notification** for every **new booking** so that I can promptly manage my schedule	Processing and Response
As a user, I want the **system response time** for each task to **not exceed 3 seconds** to ensure optimal performance	Processing and Response
As a user, I want to **book a venue online** independently without needing direct phone confirmations so that the booking process is quick and convenient	Search / Lookup Venues
As a user, I want to **use an advanced search filter** (by sport type, location/radius, ratings, and price) so that I can quickly **find a high-quality venue** that fits my team's budget and is convenient to travel to	Search & Filter
As a user (either **a solo player** or **a team**), I want to **find and connect** with **other sports groups** for friendly matches so that we can improve our skills and foster sportsmanship	Sports Matchmaking
As a user, upon **accessing** the **venue details page**, I want to **view the schedule** so that I can see exactly which **time slots** are available throughout the day or week	Lookup Time slots

As a user, I want to **view comprehensive venue details**, including a list of venue-specific add-on services (such as referee hiring), so that I can make an informed booking decision	Lookup Details / Services
As a venue owner, I want to **view a monthly revenue chart** with a detailed breakdown by sport type so that I can easily analyze my business performance	Revenue Statistics
As a venue owner, I want to **view the booking schedule** along with **customer details** (name and phone number) so that I can verify the customer's identity	Verify Booking
As a user, I **want the system** to analyze my playing preferences and activity history to **provide personalized venue** and **matchmaking suggestions**	Recommendation engine
As a user, I want to use an **NLP-powered virtual assistant** (Chatbot/Voice Bot) to book a venue through natural conversation, enabling a quick and **hands-free booking experience**	NLP-powered virtual assistant
As a venue owner, I want the system to **predict** the likelihood of **a customer no-show** and dynamically adjust the required deposit amount so that I can **minimize revenue loss** and **optimize venue utilization**	Smart No-Show Prevention System

3.1.2. Software Architecture

To satisfy the functionalities listed above while keeping the system maintainable for a five-person student team, SPOT adopts a layered (N-tier) architecture in which the business logic is further organized into independent, domain-based services. This hybrid "modular-monolith with separable AI services" approach allows the core booking platform to be delivered quickly, while the two AI-driven features (recommendation engine and NLP virtual assistant) can be developed, scaled, and even redeployed independently without destabilizing the core system.



1) **Presentation Layer (Client Tier)**
   - Customer Mobile/Web App: booking, search & filter, schedule lookup, matchmaking, reviews, and the NLP virtual assistant (chat/voice).
   - Owner/Admin Web Console: schedule management, customer verification, revenue statistics, booking notifications.
   - All clients communicate over HTTPS (REST/JSON) and receive real-time updates (booking status, new-booking alerts) through WebSocket/push notifications.
2) **API Gateway Layer**

A single entry point that performs authentication/JWT validation, OTP and brute-force-attempt checks, request routing, rate limiting, and response aggregation before forwarding requests to the appropriate backend service. This is also where role-based access control (RBAC) is enforced for the three roles: Customer, Venue Owner, and Administrator.

3) **Application/Business Logic Layer (Backend Services)**
   - **Auth & Identity Service –** registration/login, OTP re-confirmation for sensitive actions, brute-force lockout, JWT issuance and RBAC policy storage.
   - **Venue & Catalog Service –** venue profiles, add-on services, advanced search/filter (sport type, location/radius, rating, price).
   - **Booking & Scheduling Service –** time-slot lookup, reservation, conflict checking, booking modification/cancellation by owners/admins; guarantees the < 3-second response target via slot caching.
   - **Matchmaking Service –** connects solo players/teams for friendly matches based on sport, level, and location.
   - **Payment & Deposit Service –** integrates third-party payment gateways and applies the dynamic deposit amount produced by the No-Show Prevention model.
   - **Notification Service –** sends booking/new-booking, OTP, and reminder messages via push, email, and SMS, triggered asynchronously through a message queue.
   - **Review & Rating Service –** collects and moderates post-booking venue reviews/ratings.
   - **Analytics & Revenue Reporting Service –** aggregates booking data into the monthly revenue chart broken down by sport type.
   - **Recommendation Engine (AI service) –** a content-based/collaborative-filtering model that ranks venues and matchmaking suggestions from user activity history.
   - **NLP Virtual Assistant Service (AI service) –** speech-to-text and intent recognition (using a large language model API), translating natural conversation into Booking Service calls for a hands-free flow.
   - **Smart No-Show Prevention Service (AI service) –** a classification model estimating no-show probability per booking and feeding the result into the Payment Service to adjust the deposit amount dynamically.

4) Data Layer

A relational database (PostgreSQL/MySQL) stores transactional data (users, venues, bookings, payments, reviews); an in-memory cache (Redis) holds frequently accessed time-slot and session data to keep response time under 3 seconds; object storage (e.g., AWS S3/Firebase Storage) holds venue images and media; usage logs feed the Analytics service and, in turn, the Recommendation and No-Show models.

5) Integration & Cross-Cutting Concerns

External integrations include the SMS/OTP gateway, payment gateway (e.g., VNPay/MoMo), maps/geolocation API for radius search, and the LLM API used by the virtual assistant. Cross-cutting concerns – HTTPS/TLS, OAuth2/JWT authentication, centralized logging and monitoring, and a lightweight message broker (RabbitMQ/Kafka) for asynchronous events (notifications, analytics, AI inference requests) – are applied uniformly across all services. Services communicate synchronously via REST for request/response operations and asynchronously via the message broker for events that do not block the user-facing response, which is key to meeting the under-3-second performance requirement.

3.2. Hardware

As a cloud-based web/mobile platform, SPOT does not require any specialized or proprietary equipment. The hardware needs fall into three groups: server-side (hosting) infrastructure, client-side end-user devices, and the team's development machines.

1) Server-side / Cloud Hosting Infrastructure

The system will be deployed on a public cloud provider rather than on physical, team-owned servers, so that capacity can scale with the number of bookings. To align with our zero-cost budget for the initial development phase, the team will thoroughly leverage the GitHub Student Developer Pack. By utilizing the free cloud credits provided in this pack (e.g., for Digital Ocean or Microsoft Azure), we can provision a robust infrastructure without incurring upfront costs.

Component	Recommended Specification	Purpose
Application server (API Gateway + backend services)	2–4 vCPU, 4–8 GB RAM, Linux (Ubuntu), containerized (Docker)	Runs core booking, auth, payment, matchmaking and notification services
Database server (managed)	2 vCPU, 4 GB RAM, 20–50 GB SSD, automated daily backup	PostgreSQL/MySQL instance for transactional data
Cache server	1 vCPU, 1–2 GB RAM (Redis)	Time-slot/session caching to keep response time < 3s
Object/media storage	Cloud bucket storage (e.g., AWS S3/Firebase Storage), starting at 10–20 GB	Venue images and other media assets
AI/ML inference (recommendation, NLP, no-show)	Shared 2 vCPU/4 GB instance, or pay-per-call third-party LLM API (no dedicated GPU required initially)	Hosts the AI microservices; can scale to GPU instances later if self-hosted models are required

2) Client-side (End-user) Devices

- Customers/players: a smartphone running Android 8.0+ or iOS 13+ (for the mobile app), or any PC/laptop with a modern browser (Chrome, Edge, Firefox, Safari) for the web app.
- Venue owners/administrators: a PC/laptop with a modern browser to access the management web console; a smartphone is optional for receiving push notifications on the go.
- Network: a stable internet connection (Wi-Fi, 4G/5G, or broadband) is required by all parties; the system itself does not require any dedicated network hardware on the client side.

3) Development & Testing Equipment

- Each team member uses a personal laptop/PC with at least an Intel Core i5 (or equivalent) or AMD Ryzen 5, 16 GB RAM, and an SSD to comfortably run the local development environment, Docker containers, and emulators.
- At least one Android device and one iOS device (physical or emulator/simulator) are needed for cross-platform mobile testing.

4 Development Plan

4.1. Requirements Analysis

4.1.1. Objectives

- Clarify the project objectives and stakeholder expectations
- To comprehensively define, analyze, and document the functional requirements (e.g., booking, matchmaking, AI services) and non-functional constraints (e.g., sub-3-second response time, strict security protocols) of the SPOT system
- Define the project scope, key features, required resources, and a preliminary estimate of the implementation timeline

4.1.2. Stages of Requirements Analysis

- **Elicitation & Initiation:** Gather detailed needs from potential users (solo players, teams) and venue owners to establish core business rules
- **Analysis & Modeling:** Define the system's role-based access control (Customer, Venue Owner, Administrator) and outline the logic for the Recommendation engine and Smart No-Show Prevention System
- **Specification:** Document all analyzed data into formalized requirements

4.1.3. Deliverables

- Vision documents and use cases
- Overall development plan, assignment of tasks and roles for each member

4.1.4. Time

From **June 21, 2025** to **July 4, 2025** (2 Weeks)

4.2. Software Design

4.2.1. Objectives

- Develop a clear, scalable software design that meets the specific business requirements
- The design aims to separate the floors, ensuring ease of maintenance and access. Easy to integrate with the backend in the future

4.2.2. Software Architecture Design

To satisfy the functionalities listed above while keeping the system maintainable for a student team, SPOT adopts a layered (N-tier) architecture. The business logic is organized into independent, domain-based services using a hybrid "modular-monolith with separable AI services" approach. This allows the core booking platform to be delivered quickly, while the AI-driven features can be developed and scaled independently without destabilizing the core system

1) **Presentation Layer (Client Tier):**
   - **Customer Mobile/Web App:** Built with React Native and React + Next.js, handling booking, search & filter, schedule lookup, matchmaking, reviews, and the NLP virtual assistant
   - **Owner/Admin Web Console:** Built with React + Tailwind for schedule management, customer verification, revenue statistics, and automated notifications
   - **Communication***:* All clients communicate over HTTPS (REST/JSON) and receive real-time updates via WebSocket/push notifications
2) **API Gateway Layer:**
   - Powered by Node.js (Express) and NGINX, acting as a single entry point
   - Performs authentication/JWT validation, OTP checks, brute-force-attempt prevention, request routing, rate limiting, and enforces role-based access control (RBAC)
3) **Application/Business Logic Layer (Backend Services):**
   - **Core Services (Node.js):** Includes Auth & Identity, Venue & Catalog, Booking & Scheduling (guarantees < 3-second response via slot caching), Matchmaking, Payment & Deposit, Notification, Review & Rating, and Analytics
   - AI-Powered Services (Python & LLM):
     - *Recommendation Engine:* A content-based/collaborative-filtering model using Python and scikit-learn
     - *Smart No-Show Prevention:* A classification model using Python and XGBoost to dynamically adjust deposit amounts
     - *NLP Virtual Assistant:* Utilizes Gemini / GPT APIs for speech-to-text and intent recognition

4) **Data Layer:**
   - **PostgreSQL:** Stores transactional data including users, venues, bookings, and payments
   - **Redis:** An in-memory cache holding frequently accessed time-slot and session data to maintain high performance
   - **AWS S3:** Object storage for venue images and media
5) **External Integrations:**
   - The system seamlessly integrates with VNPay/MoMo for payments, Twilio/Firebase for notifications, Maps API for location searches, and external LLM APIs for virtual assistance

4.2.3. User Interface Design (UI Design)

- Develop a user-friendly, clear, and easy-to-use interface for transaction staff and management
- Tools: Figma (for Student)

4.2.4. Database Design

To establish a secure, scalable, and highly available data layer that ensures strict data integrity for business transactions while meeting the system's sub-3-second response time requirement

1) **Relational Database (PostgreSQL/MySQL)**
   - Acts as the primary storage for all structured and transactional data. It ensures ACID properties (Atomicity, Consistency, Isolation, Durability) which are critical for the booking and payment lifecycle
   - **Core Entities:** Users (including role-based attributes for Customers, Owners, and Admins), Venues (profiles and add-on services), Bookings, Payments, and Reviews/Ratings
2) **In-Memory Cache (Redis)**
   - Deployed to handle high-frequency read operations and reduce the load on the primary relational database
   - **Usage:** Temporarily stores frequently accessed time-slots and active user sessions. This caching mechanism is the key technical enabler for guaranteeing the < 3-second system response target during peak booking hours
3) **Object/Media Storage (AWS S3 / Firebase Storage)**
   - Dedicated to handling unstructured, large media files
   - **Usage:** Efficiently stores and serves venue images, promotional banners, and other media assets, ensuring fast load times on the client side

4) Data Pipeline for AI Integration

- Transactional data and usage logs (such as booking history and user activity) are continuously aggregated to feed the AI microservices. This historical data acts as the training and inference foundation for the Recommendation Engine and the Smart No-Show classification models

4.2.5. Deliverables

- System Architectural Design Documents describing the overall structure form, main components, and data flow
- Complete UI design file in Figma
- ERD and RDM models, and scripts for creating database tables

4.2.6. Time

From **July 5, 2025** to **July 18, 2025** (2 Weeks)

4.3. Implementation

4.3.1. Objectives

- Proceed with building a complete system, fully integrating all components. The frontend and backend are based on the completed design
- Complete the programming of core system functions
- Ensure that the modules operate stably and that data is accurately exchanged between the user interface and the database

4.3.2. Deliverables

- The **SPOT** system is fully operational and meets all the stated functional requirements
- The source code repository is stored and managed on GitHub.

4.3.3. Time

From **July 19, 2025** to **August 15, 2025** (4 Weeks)

4.4. Testing

4.4.1. Objectives

- Conduct thorough testing to ensure the system is stable, error-free, and user-friendly
- Verify the completeness of the implemented features and improve functionality based on test results
- Present the product to customers, gather feedback, and make modifications as needed

4.4.2. Deliverables

- A complete **SPOT** system, tested and fine-tuned
- A detailed test documentation package

4.4.3. Time

From **August 16, 2025** to **August 22, 2025** (1 Week)

4.5. Deployment and Maintenance

4.5.1. Objectives

- Ensure the smooth and automated integration of the **SPOT** product into the official operating environment, minimizing the risk of service disruption
- After deployment, the maintenance process aims to keep the system operating stably and securely, while quickly resolving any issues that arise and updating necessary patches, ensuring that the software always effectively meets user needs

4.5.2. Deliverables

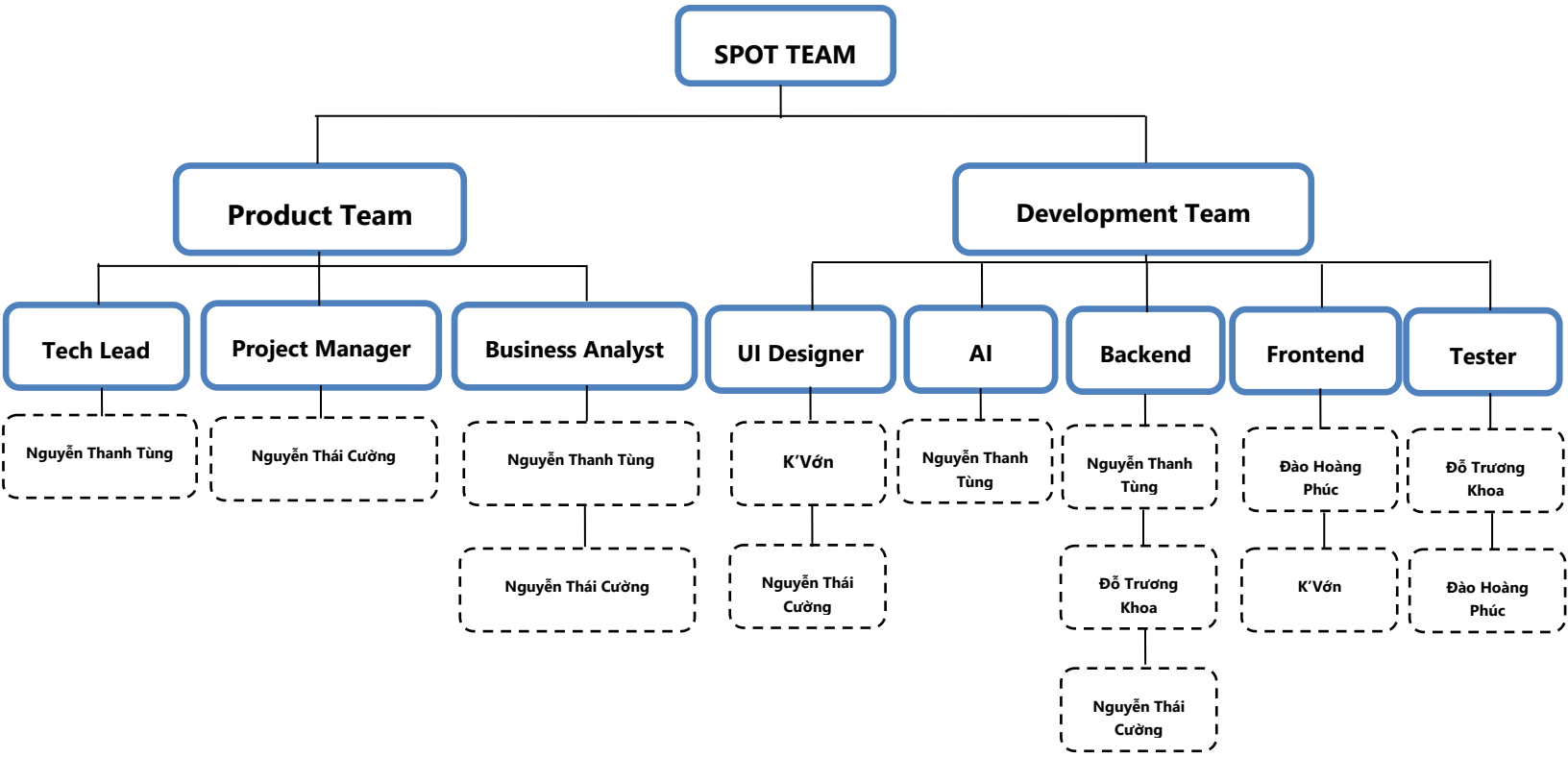
- The **SPOT** system website is currently running with a stable version installed on server
- User feedback forms and review summaries
- Detailed product upgrade and maintenance plan

4.5.3. Time

From **August 23, 2025** to **August 29, 2025** (1 Week)

5 Human Resources & Costing Plan

5.1. Human Recourses Plan



5.2. Costing Plan

5.2.1. Cost Estimate

Category	Detail	Estimate
Hosting + DB Cloud	Vercel	600.000 VNĐ
Domain name (year)	spot.id.vn	60.000 VNĐ
Maintenance and Operating	Monitor log, Check performance, Fix bug after operation	0 VNĐ
Human Resources	The dedication of our members to elevating the design and development capabilities of the SPOT team	0 VNĐ
Software and Development tools	VS code, GitHub, Figma, Jira, Gemini (Pro), ChatGPT	0 VNĐ
Total		660.000 VNĐ

5.2.2. Time

Category	Estimate
Initiation & Requirements analysis & Planning	2 Weeks
Design Software	2 Weeks
Implementation / Execution	4 Weeks
Testing	1 Week
Evaluation / Deployment & Maintenance	1 Week
Total	10 Weeks

6 Tools setup

- One of the main goals of this project assignment is to help our team practice teamwork for professional software development. We understand that using tools effectively to support teamwork is an important criterion for grading, and everyone is required to work closely with others to deliver results
- To ensure seamless collaboration and project success, the SPOT team will utilize the following stack of tools:

1) **Course Management**
   - **Moodle:** Used for posting and submitting assignments as required by the instructors
2) **Communication & Collaboration**
   - **Discord:** Used for daily discussions and interactions among group members. The team will utilize the general channel for overall project discussions and a specific channel linked to our project management board for automated updates. Members are required to check the desktop and mobile applications frequently
3) **Project Management**
   - **Jira:** Used as the primary agile project management tool. The team will have one centralized board where tasks are systematically organized into sprints
4) **Design & AI Assistance**
   - **Figma:** Utilized by the UI/UX designers to create wireframes, prototypes, and the complete UI design file
   - **Gemini (Pro) & ChatGPT:** Used as AI coding assistants to research solutions, optimize algorithms, and assist the AI-Powered Services development
5) **Development & Version Control**
   - **VS Code:** The primary Integrated Development Environment (IDE) chosen by the development team
   - **GitHub:** Used as the source control tool to collaboratively store source code and project documentation

- Repository Structure: Our GitHub repository will strictly follow the required structure to organize the team's assets
  - /src: Used to store all the source code for the backend, frontend, and AI services
  - /docs: Used to store project documentation
  - /pa: Including subfolders to store Project Assignment submissions